Java tricky questions

Question 1. Break Statement Outside Loop

```
public class A {
  public static void main(String[] args)
  {
    if (true)
      break;
  }
}
```

Choices:

- a) Nothing
- b) Error

Answer: b) Error

Explanation:

- The break statement is valid only inside a loop or a switch block.
- Using break inside an if statement causes a compile-time error:"**break outside switch or loop**".

Question 2. Character Addition vs String Concatenation

```
public class A {
  public static void main(String[] args)
  {
    System.out.println('j' + 'a' + 'v' + 'a');
  }
}
```

Choices:

- a) java
- b) Something else (Other than simple concatenation)

Answer: b) Something else (Other than simple concatenation)

Explanation:

- Character literals ('j', 'a', 'v', 'a') are treated as char, not String.
- The + operator performs numeric addition on character Unicode values.
- Calculation:106 + 97 + 118 + 97 = 418
- String concatenation would occur only with double-quoted literals.

Question 3. Valid Java Identifiers

```
public class A {
  public static void main(String[] args)
  {
    int $_ = 5;
  }
}
```

Choices:

- a) Nothing
- b) Error

Answer: a) Nothing

Explanation:

- In Java, identifiers can start with: Letters, Underscore (_), Dollar sign ($)
- The variable name $_ is valid, so the program compiles and runs without output.

Question 4. Integer Caching and Reference Comparison

```
public class Demo{
  public static void main(String[] arr){
    Integer num1 = 100;
    Integer num2 = 100;
    Integer num3 = 500;
    Integer num4 = 500;
```

```
    if(num1==num2){
      System.out.println("num1 == num2");
    }
    else{
      System.out.println("num1 != num2");
    }
    if(num3 == num4){
      System.out.println("num3 == num4");
    }
    else{
      System.out.println("num3 != num4");
    }
  }
}
```

Choices:

- a) num1 == num2 num3 == num4
- b) num1 == num2 num3 != num4
- c) num1 != num2 num3 == num4
- d) num1 != num2 num3 != num4

Answer: b) num1 == num2 num3 != num4

Explanation:

- Java caches **Integer** objects in the range **-128 to 127.**
- Autoboxed values within this range reference the same object.
- 100 is cached → same reference → == returns true.
- 500 is not cached → different objects → == returns false.
- == compares object references, not values.

Question 5. Overloading the main() Method

```
public class Demo {
```

```
    public static void main(String[] arr) {}

    public static void main(String arr) {}
}
```

Choices:

- a) Nothing
- b) Error

Answer: a) Nothing

Explanation:

- The **main()** method can be overloaded.
- The JVM only invokes main(String[] args) as the entry point.
- The overloaded main(String) method is ignored unless explicitly called.